Module 35

Application Design and Development/5: Application Development and Mobile

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

March 17, 2026

Module Recap

- Learnt how to accessing PostgreSQL database from Python

Module Recap

- Learnt how to accessing PostgreSQL database from Python
- Learnt to build Python Web Application with PostgreSQL and Flask

Module Objectives

Goals

- To explore the Rapid Application Development Process

Module Objectives

Goals

- To explore the Rapid Application Development Process
- To understand the issues in Application Performance and Security

Module Objectives

Goals

- To explore the Rapid Application Development Process
- To understand the issues in Application Performance and Security
- To understand the similarity and differences between Mobile Apps and Web Applications

Module Outline

- Rapid Application Development

Module Outline

- Rapid Application Development
- Application Performance and Security

Module Outline

- Rapid Application Development
- Application Performance and Security
- Mobile Apps

Rapid Application Development

Rapid Application Development

- A lot of effort is required to develop Web application interfaces, especially the rich interaction functionality associated with Web 2.0 applications

Rapid Application Development

- A lot of effort is required to develop Web application interfaces, especially the rich interaction functionality associated with Web 2.0 applications
- Several approaches to speed up application development

Rapid Application Development

- A lot of effort is required to develop Web application interfaces, especially the rich interaction functionality associated with Web 2.0 applications
- Several approaches to speed up application development
  - Function library to generate user-interface elements

Rapid Application Development

- A lot of effort is required to develop Web application interfaces, especially the rich interaction functionality associated with Web 2.0 applications
- Several approaches to speed up application development
  - Function library to generate user-interface elements
  - Drag-and-drop features in an IDE to create user-interface elements

Rapid Application Development

- A lot of effort is required to develop Web application interfaces, especially the rich interaction functionality associated with Web 2.0 applications
- Several approaches to speed up application development
  - Function library to generate user-interface elements
  - Drag-and-drop features in an IDE to create user-interface elements
  - Automatically generate code for user interface from a declarative specification

Rapid Application Development

- A lot of effort is required to develop Web application interfaces, especially the rich interaction functionality associated with Web 2.0 applications
- Several approaches to speed up application development
  - Function library to generate user-interface elements
  - Drag-and-drop features in an IDE to create user-interface elements
  - Automatically generate code for user interface from a declarative specification
- RAD Software is an agile model that focuses on fast prototyping and quick feedback

Rapid Application Development

- A lot of effort is required to develop Web application interfaces, especially the rich interaction functionality associated with Web 2.0 applications
- Several approaches to speed up application development
  - Function library to generate user-interface elements
  - Drag-and-drop features in an IDE to create user-interface elements
  - Automatically generate code for user interface from a declarative specification
- RAD Software is an agile model that focuses on fast prototyping and quick feedback
- App development has 4 phases: business modeling, data modeling, process modeling, and testing & turnover

Rapid Application Development

- A lot of effort is required to develop Web application interfaces, especially the rich interaction functionality associated with Web 2.0 applications
- Several approaches to speed up application development
  - Function library to generate user-interface elements
  - Drag-and-drop features in an IDE to create user-interface elements
  - Automatically generate code for user interface from a declarative specification
- RAD Software is an agile model that focuses on fast prototyping and quick feedback
- App development has 4 phases: business modeling, data modeling, process modeling, and testing & turnover
- With RAD, the time between prototypes and iterations is short, and integration occurs since inception

Rapid Application Development (2)

- **Web application development frameworks**

Rapid Application Development (2)

- **Web application development frameworks**
  - **Java Server Faces (JSF):** APIs for representing UI components, managing state, handling events, input validation, and page navigation

Rapid Application Development (2)

- **Web application development frameworks**
  - **Java Server Faces (JSF):** APIs for representing UI components, managing state, handling events, input validation, and page navigation
  - **Ruby on Rails:** Easy creation of CRUD interfaces by code generation from database schema or object model

Rapid Application Development (2)

- **Web application development frameworks**
  - **Java Server Faces (JSF):** APIs for representing UI components, managing state, handling events, input validation, and page navigation
  - **Ruby on Rails:** Easy creation of CRUD interfaces by code generation from database schema or object model
- **RAD Platforms and Tools**

Rapid Application Development (2)

- **Web application development frameworks**
  - **Java Server Faces (JSF):** APIs for representing UI components, managing state, handling events, input validation, and page navigation
  - **Ruby on Rails:** Easy creation of CRUD interfaces by code generation from database schema or object model
- **RAD Platforms and Tools**
  - G Suite, Google App Engine

Rapid Application Development (2)

- **Web application development frameworks**
  - **Java Server Faces (JSF):** APIs for representing UI components, managing state, handling events, input validation, and page navigation
  - **Ruby on Rails:** Easy creation of CRUD interfaces by code generation from database schema or object model
- **RAD Platforms and Tools**
  - G Suite, Google App Engine
  - Microsoft Azure, Amazon EC2, AWS Elastic Beanstalk

ASP.NET and Visual Studio

- ASP.NET provides a variety of controls that are interpreted at server, and generate HTML code

ASP.NET and Visual Studio

- ASP.NET provides a variety of controls that are interpreted at server, and generate HTML code
- Visual Studio provides drag-and-drop development using these controls

ASP.NET and Visual Studio

- ASP.NET provides a variety of controls that are interpreted at server, and generate HTML code
- Visual Studio provides drag-and-drop development using these controls
  - For example, menus and list boxes can be associated with `DataSet` object

ASP.NET and Visual Studio

- ASP.NET provides a variety of controls that are interpreted at server, and generate HTML code
- Visual Studio provides drag-and-drop development using these controls
  - For example, menus and list boxes can be associated with `DataSet` object
- Validator controls (constraints) can be added to form input fields

ASP.NET and Visual Studio

- ASP.NET provides a variety of controls that are interpreted at server, and generate HTML code
- Visual Studio provides drag-and-drop development using these controls
  - For example, menus and list boxes can be associated with `DataSet` object
- Validator controls (constraints) can be added to form input fields
  - JavaScript to enforce constraints at client, and separately enforced at server

ASP.NET and Visual Studio

- ASP.NET provides a variety of controls that are interpreted at server, and generate HTML code
- Visual Studio provides drag-and-drop development using these controls
  - For example, menus and list boxes can be associated with `DataSet` object
- Validator controls (constraints) can be added to form input fields
  - JavaScript to enforce constraints at client, and separately enforced at server
- User actions such as selecting a value from a menu can be associated with actions at server

ASP.NET and Visual Studio

- ASP.NET provides a variety of controls that are interpreted at server, and generate HTML code
- Visual Studio provides drag-and-drop development using these controls
  - For example, menus and list boxes can be associated with `DataSet` object
- Validator controls (constraints) can be added to form input fields
  - JavaScript to enforce constraints at client, and separately enforced at server
- User actions such as selecting a value from a menu can be associated with actions at server
- DataGrid provides convenient way of displaying SQL query results in tabular format

Application Performance and Security

Application Performance

- Performance is an issue for popular Web sites

Application Performance

- Performance is an issue for popular Web sites
  - May be accessed by millions of users every day, thousands of requests per second at peak time

Application Performance

- Performance is an issue for popular Web sites
  - May be accessed by millions of users every day, thousands of requests per second at peak time
- Caching techniques used to reduce cost of serving pages by exploiting commonalities between requests

Application Performance

- Performance is an issue for popular Web sites
  - May be accessed by millions of users every day, thousands of requests per second at peak time
- Caching techniques used to reduce cost of serving pages by exploiting commonalities between requests
- At the server site:

Application Performance

- Performance is an issue for popular Web sites
  - May be accessed by millions of users every day, thousands of requests per second at peak time
- Caching techniques used to reduce cost of serving pages by exploiting commonalities between requests
- At the server site:
  - Caching of JDBC connections between servlet requests (connection pooling)

Application Performance

- Performance is an issue for popular Web sites
  - May be accessed by millions of users every day, thousands of requests per second at peak time
- Caching techniques used to reduce cost of serving pages by exploiting commonalities between requests
- At the server site:
  - Caching of JDBC connections between servlet requests (connection pooling)
  - Caching results of database queries (must be updated if underlying database changes)

Application Performance

- Performance is an issue for popular Web sites
  - May be accessed by millions of users every day, thousands of requests per second at peak time
- Caching techniques used to reduce cost of serving pages by exploiting commonalities between requests
- At the server site:
  - Caching of JDBC connections between servlet requests (connection pooling)
  - Caching results of database queries (must be updated if underlying database changes)
  - Caching of generated HTML

Application Performance

- Performance is an issue for popular Web sites
  - May be accessed by millions of users every day, thousands of requests per second at peak time
- Caching techniques used to reduce cost of serving pages by exploiting commonalities between requests
- At the server site:
  - Caching of JDBC connections between servlet requests (connection pooling)
  - Caching results of database queries (must be updated if underlying database changes)
  - Caching of generated HTML
- At the client's network

Application Performance

- Performance is an issue for popular Web sites
  - May be accessed by millions of users every day, thousands of requests per second at peak time
- Caching techniques used to reduce cost of serving pages by exploiting commonalities between requests
- At the server site:
  - Caching of JDBC connections between servlet requests (connection pooling)
  - Caching results of database queries (must be updated if underlying database changes)
  - Caching of generated HTML
- At the client's network
  - Caching of pages by Web proxy

Application Security: SQL Injection

- Suppose query is constructed using

```
"select * from instructor where name = '" + name + "'"
```

- Suppose the user, instead of entering a name, enters:

```
X' or 'Y' = 'Y
```

- then the resulting statement becomes:

```
"select * from instructor where name = '" + "X' or 'Y' = 'Y" + "'"
```

- which is:
  - ▷ `select * from instructor where name = 'X' or 'Y' = 'Y'`
- User could have even used
  - ▷ `X'; update instructor set salary = salary + 10000; --`
- Prepared statement internally uses:

```
select * from instructor where name = 'X \' or \'Y \' = \'Y'
```

- **Always use prepared statements, with user inputs as parameters**
- Is the following prepared statement secure?

```
conn.prepareStatement("select * from instructor where name = '" + name
                      + "'")
```

Application Security (2): Password Leakage

Warning

Never store passwords in clear text in scripts that may be accessible to users

- For example, in files in a directory accessible to a web server

Application Security (2): Password Leakage

Warning

Never store passwords in clear text in scripts that may be accessible to users

- For example, in files in a directory accessible to a web server
  - Web server will execute but not provide source of script files such as `file.jsp`, but source of editor backup files such as `file.jsp~` may be served

Application Security (2): Password Leakage

Warning

Never store passwords in clear text in scripts that may be accessible to users

- For example, in files in a directory accessible to a web server
  - Web server will execute but not provide source of script files such as `file.jsp`, but source of editor backup files such as `file.jsp~` may be served
- Restrict access to database server from IPs of machines running application servers

Application Security (2): Password Leakage

Warning

Never store passwords in clear text in scripts that may be accessible to users

- For example, in files in a directory accessible to a web server
  - Web server will execute but not provide source of script files such as `file.jsp`, but source of editor backup files such as `file.jsp~` may be served
- Restrict access to database server from IPs of machines running application servers
  - Most databases allow restriction of access by source IP address

Application Security (3): Authentication

- Single factor authentication such as passwords too risky for critical applications

Application Security (3): Authentication

- Single factor authentication such as passwords too risky for critical applications
  - Vulnerable to: guessing, packet sniffing, password reuse across sites, spyware

Application Security (3): Authentication

- Single factor authentication such as passwords too risky for critical applications
  - Vulnerable to: guessing, packet sniffing, password reuse across sites, spyware
- **Two-factor authentication**

Application Security (3): Authentication

- Single factor authentication such as passwords too risky for critical applications
  - Vulnerable to: guessing, packet sniffing, password reuse across sites, spyware
- **Two-factor authentication**
  - Password plus one-time password sent by SMS

Application Security (3): Authentication

- Single factor authentication such as passwords too risky for critical applications
  - Vulnerable to: guessing, packet sniffing, password reuse across sites, spyware
- **Two-factor authentication**
  - Password plus one-time password sent by SMS
  - Password plus hardware one-time password device

Application Security (3): Authentication

- Single factor authentication such as passwords too risky for critical applications
  - Vulnerable to: guessing, packet sniffing, password reuse across sites, spyware
- **Two-factor authentication**
  - Password plus one-time password sent by SMS
  - Password plus hardware one-time password device
    - device generates a new pseudo-random number every minute

Application Security (3): Authentication

- Single factor authentication such as passwords too risky for critical applications
  - Vulnerable to: guessing, packet sniffing, password reuse across sites, spyware
- **Two-factor authentication**
  - Password plus one-time password sent by SMS
  - Password plus hardware one-time password device
    - device generates a new pseudo-random number every minute
    - application server generates same sequence to verify

Application Security (4): Application-Level Authorization

- Current SQL standard does not allow fine-grained authorization such as 'students can see their own grades, but not other's grades'
- **Problem 1:** Database has no idea who are application users
- **Problem 2:** SQL authorization is at the level of tables, or columns of tables, but not to specific rows of a table
- One workaround: use views such as

```
create view studentTakes as
select * from takes
where takes.ID = syscontext.user_id()
```

- where `syscontext.user_id()` provides end user identity
  - ▷ end user identity must be provided to the database by the application
- Having multiple such views is cumbersome

Application Security (5): Application-Level Authorization

- Currently, authorization is done entirely in application

Application Security (5): Application-Level Authorization

- Currently, authorization is done entirely in application
  - Entire application code has access to entire database – large surface area, making protection harder

Application Security (5): Application-Level Authorization

- Currently, authorization is done entirely in application
  - Entire application code has access to entire database – large surface area, making protection harder
- Alternative: fine-grained (row-level) authorization schemes

Application Security (5): Application-Level Authorization

- Currently, authorization is done entirely in application
  - Entire application code has access to entire database – large surface area, making protection harder
- Alternative: fine-grained (row-level) authorization schemes
  - Extensions to SQL authorization proposed but not currently implemented

Application Security (5): Application-Level Authorization

- Currently, authorization is done entirely in application
  - Entire application code has access to entire database – large surface area, making protection harder
- Alternative: fine-grained (row-level) authorization schemes
  - Extensions to SQL authorization proposed but not currently implemented
  - Oracle Virtual Private Database (VPD) allows predicates to be added transparently to all SQL queries

Application Security (6): Audit Trails

- Applications must log actions to an audit trail, to detect who carried out an update, or accessed some sensitive data

Application Security (6): Audit Trails

- Applications must log actions to an audit trail, to detect who carried out an update, or accessed some sensitive data
- Audit trails used after-the-fact to

Application Security (6): Audit Trails

- Applications must log actions to an audit trail, to detect who carried out an update, or accessed some sensitive data
- Audit trails used after-the-fact to
  - detect security breaches

Application Security (6): Audit Trails

- Applications must log actions to an audit trail, to detect who carried out an update, or accessed some sensitive data
- Audit trails used after-the-fact to
  - detect security breaches
  - repair damage caused by security breach

Application Security (6): Audit Trails

- Applications must log actions to an audit trail, to detect who carried out an update, or accessed some sensitive data
- Audit trails used after-the-fact to
  - detect security breaches
  - repair damage caused by security breach
  - trace who carried out the breach

Application Security (6): Audit Trails

- Applications must log actions to an audit trail, to detect who carried out an update, or accessed some sensitive data
- Audit trails used after-the-fact to
  - detect security breaches
  - repair damage caused by security breach
  - trace who carried out the breach
- Audit trails needed at

Application Security (6): Audit Trails

- Applications must log actions to an audit trail, to detect who carried out an update, or accessed some sensitive data
- Audit trails used after-the-fact to
  - detect security breaches
  - repair damage caused by security breach
  - trace who carried out the breach
- Audit trails needed at
  - Database level, and at

Application Security (6): Audit Trails

- Applications must log actions to an audit trail, to detect who carried out an update, or accessed some sensitive data
- Audit trails used after-the-fact to
  - detect security breaches
  - repair damage caused by security breach
  - trace who carried out the breach
- Audit trails needed at
  - Database level, and at
  - Application level

Challenges in Web Application Development

Challenges in Web Application Development

- User Interface and User Experience

Source: 5 Challenges in Web Application Development

Challenges in Web Application Development

- User Interface and User Experience
- Scalability

Source: 5 Challenges in Web Application Development

Challenges in Web Application Development

- User Interface and User Experience
- Scalability
- Performance

Source: 5 Challenges in Web Application Development

Challenges in Web Application Development

- User Interface and User Experience
- Scalability
- Performance
- Knowledge of Framework and Platforms

Source: 5 Challenges in Web Application Development

Challenges in Web Application Development

- User Interface and User Experience
- Scalability
- Performance
- Knowledge of Framework and Platforms
- Security

Source: 5 Challenges in Web Application Development

Mobile Apps

What is a Mobile App?

- A type of application software designed to run on a mobile device, such as a smartphone or tablet computer

What is a Mobile App?

- A type of application software designed to run on a mobile device, such as a smartphone or tablet computer
- Designed with consideration for device demands and constraints, and to take advantage of specialized capabilities

What is a Mobile App?

- A type of application software designed to run on a mobile device, such as a smartphone or tablet computer
- Designed with consideration for device demands and constraints, and to take advantage of specialized capabilities
- **Constraints:**

What is a Mobile App?

- A type of application software designed to run on a mobile device, such as a smartphone or tablet computer
- Designed with consideration for device demands and constraints, and to take advantage of specialized capabilities
- **Constraints:**
  - Form Factor – influences display and navigation

What is a Mobile App?

- A type of application software designed to run on a mobile device, such as a smartphone or tablet computer
- Designed with consideration for device demands and constraints, and to take advantage of specialized capabilities
- **Constraints:**
  - Form Factor – influences display and navigation
  - Limited Memory, Computing Power, and Battery

What is a Mobile App?

- A type of application software designed to run on a mobile device, such as a smartphone or tablet computer
- Designed with consideration for device demands and constraints, and to take advantage of specialized capabilities
- **Constraints:**
  - Form Factor – influences display and navigation
  - Limited Memory, Computing Power, and Battery
  - Limited Bandwidth

What is a Mobile App?

- A type of application software designed to run on a mobile device, such as a smartphone or tablet computer
- Designed with consideration for device demands and constraints, and to take advantage of specialized capabilities
- **Constraints:**
  - Form Factor – influences display and navigation
  - Limited Memory, Computing Power, and Battery
  - Limited Bandwidth
- **Advantages:**

What is a Mobile App?

- A type of application software designed to run on a mobile device, such as a smartphone or tablet computer
- Designed with consideration for device demands and constraints, and to take advantage of specialized capabilities
- **Constraints:**
  - Form Factor – influences display and navigation
  - Limited Memory, Computing Power, and Battery
  - Limited Bandwidth
- **Advantages:**
  - Availability of sensors like accelerometer

What is a Mobile App?

- A type of application software designed to run on a mobile device, such as a smartphone or tablet computer
- Designed with consideration for device demands and constraints, and to take advantage of specialized capabilities
- **Constraints:**
  - Form Factor – influences display and navigation
  - Limited Memory, Computing Power, and Battery
  - Limited Bandwidth
- **Advantages:**
  - Availability of sensors like accelerometer
  - Touchscreen – Gesture-based Navigation

Mobile Website vis-à-vis Mobile App

Mobile Website

- Browser-based HTML pages
- Accessed over WiFi/3G/4G
- Designed for small touch-screen
- Can use click-to-call and location mapping

Mobile Apps

- Downloaded and installed on device
- From App Store, Google Play, etc.
- Can pull content from Internet or work offline

Architecture of Mobile App

- Typically **3-tier architecture**

Architecture of Mobile App

- Typically **3-tier architecture**
  - Presentation, Business, Data layers

Architecture of Mobile App

- Typically **3-tier architecture**
  - Presentation, Business, Data layers
- Data Layer is often split between:

Architecture of Mobile App

- Typically **3-tier architecture**
  - Presentation, Business, Data layers
- Data Layer is often split between:
  - Local Data (on-device storage)

Architecture of Mobile App

- Typically **3-tier architecture**
  - Presentation, Business, Data layers
- Data Layer is often split between:
  - Local Data (on-device storage)
  - Remote Data (server/cloud)

Architecture of Mobile App

- Typically **3-tier architecture**
  - Presentation, Business, Data layers
- Data Layer is often split between:
  - Local Data (on-device storage)
  - Remote Data (server/cloud)
- Needs customization for platform: Android, iOS, Windows

Types of Mobile Apps

- **Native Apps**: Completely written in the native language of a platform

Source: https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save

Types of Mobile Apps

- **Native Apps**: Completely written in the native language of a platform
  - iOS → Objective-C; Android → Java or C/C++

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Types of Mobile Apps

- **Native Apps**: Completely written in the native language of a platform
  - iOS → Objective-C; Android → Java or C/C++
  - Platform specific (heavily dependent on OS)

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Types of Mobile Apps

- **Native Apps**: Completely written in the native language of a platform
  - iOS → Objective-C; Android → Java or C/C++
  - Platform specific (heavily dependent on OS)
- **Web Apps**: Run completely inside of a Web browser

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Types of Mobile Apps

- **Native Apps**: Completely written in the native language of a platform
  - iOS → Objective-C; Android → Java or C/C++
  - Platform specific (heavily dependent on OS)
- **Web Apps**: Run completely inside of a Web browser
  - Interfaces built with HTML/CSS, powered by Ruby on Rails, JavaScript, PHP, or Python

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Types of Mobile Apps

- **Native Apps**: Completely written in the native language of a platform
  - iOS → Objective-C; Android → Java or C/C++
  - Platform specific (heavily dependent on OS)
- **Web Apps**: Run completely inside of a Web browser
  - Interfaces built with HTML/CSS, powered by Ruby on Rails, JavaScript, PHP, or Python
  - Portable across any phone, tablet, or computer

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Types of Mobile Apps

- **Native Apps**: Completely written in the native language of a platform
  - iOS → Objective-C; Android → Java or C/C++
  - Platform specific (heavily dependent on OS)
- **Web Apps**: Run completely inside of a Web browser
  - Interfaces built with HTML/CSS, powered by Ruby on Rails, JavaScript, PHP, or Python
  - Portable across any phone, tablet, or computer
- **Hybrid Apps**: Combines attributes of both native and Web apps

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Types of Mobile Apps

- **Native Apps**: Completely written in the native language of a platform
  - iOS → Objective-C; Android → Java or C/C++
  - Platform specific (heavily dependent on OS)
- **Web Apps**: Run completely inside of a Web browser
  - Interfaces built with HTML/CSS, powered by Ruby on Rails, JavaScript, PHP, or Python
  - Portable across any phone, tablet, or computer
- **Hybrid Apps**: Combines attributes of both native and Web apps
  - Uses common cross-platform code while tailoring native-specific attributes

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Design Issues

- Determine Device

Source: https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save

Design Issues

- Determine Device
- Note Device Resources - memory, power, speed

Source: https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save

Design Issues

- Determine Device
- Note Device Resources - memory, power, speed
- Consider Bandwidth

Source: https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save

Design Issues

- Determine Device
- Note Device Resources - memory, power, speed
- Consider Bandwidth
- Decide on Architecture Layers

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Design Issues

- Determine Device
- Note Device Resources - memory, power, speed
- Consider Bandwidth
- Decide on Architecture Layers
- Select Technology

Source: https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save

Design Issues

- Determine Device
- Note Device Resources - memory, power, speed
- Consider Bandwidth
- Decide on Architecture Layers
- Select Technology
- Define User Interface

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Design Issues

- Determine Device
- Note Device Resources - memory, power, speed
- Consider Bandwidth
- Decide on Architecture Layers
- Select Technology
- Define User Interface
- Select Navigation

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Design Issues

- Determine Device
- Note Device Resources - memory, power, speed
- Consider Bandwidth
- Decide on Architecture Layers
- Select Technology
- Define User Interface
- Select Navigation
- Maintain Flow

Source: `https://www.slideshare.net/hassandar18/architecture-of-mobile-software-applications?from_action=save`

Module Summary

Summary

- Understood the steps in the Rapid Application Development Process

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- Understood the steps in the Rapid Application Development Process
- Exposed to the issues in Application Performance and Application Security

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- Understood the steps in the Rapid Application Development Process
- Exposed to the issues in Application Performance and Application Security
- Learnt the distinctive features of Mobile Apps

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.